Министерство науки и высшего образования РФ
Федеральное государственное автономное
образовательное учреждение высшего образования
«СИБИРСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт космических и информационных технологий
институт

Программная инженерия
кафедра

ОТЧЕТ О ПРАКТИЧЕСКОЙ РАБОТЕ №4

по дисциплине «Серверное программирование»

Тема: настройка ORM и подключение к базе данных

Преподаватель _____ _____ _____
подпись, дата | инициалы, фамилия

Студент _____ _____ Mohamed Amini
номер группы, зачетной книжки | подпись, дата | инициалы, фамилия

Красноярск 2026

1 Цель и постановка задачи

Цель работы – настроить связь между серверным приложением и базой данных с помощью ORM-прослойки и протестировать эту связь стандартными запросами. В эталонном задании используются Java и Hibernate; работа выполнена на стеке Node.js и TypeScript с ORM Drizzle (альтернативный стек согласован с преподавателем).

Задание требует:

– создать проект и подключить зависимости: ORM и драйвер базы данных;

– описать настройки подключения в конфигурационном файле (в эталоне – hibernate.cfg.xml);

– создать класс DAO со статическими полями «фабрика сессий» и «сессия потока» и методами getSession(), begin() и commit();

– создать класс TestSrvs – наследник DAO – с методом getName(id), возвращающим текстовое поле записи с заданным идентификатором;

– создать точку входа Main, вызвать getName и вывести результат в консоль;

– реализовать ещё два метода: список значений одного поля из заданного промежутка идентификаторов и создание новой записи внутри пары begin() и commit().

2 Соответствие эталонного стека и использованного

Каждому элементу эталонной реализации на Java соответствует элемент реализации на TypeScript (таблица 1). Данные берутся из базы «ВелоГрад» практических заданий № 1–3; для проверки используется таблица station (станции проката).

Таблица 1 – Соответствие Java и TypeScript

Эталон (Java)	**Реализация (TypeScript)**
Maven, pom.xml	npm, package.json
Hibernate	Drizzle ORM
драйвер JDBC PostgreSQL	драйвер pg (node-postgres)

Эталон (Java)	Реализация (TypeScript)
hibernate.cfg.xml	src/config.ts
класс-сущность с аннотациями	описание таблицы pgTable в src/schema.ts
SessionFactory	пул соединений Pool (статическое поле DAO.sessionFactory)
ThreadLocal	статическое поле DAO.session (в Node.js выполняется один поток)
session.beginTransaction(), getTransaction().commit()	команды BEGIN и COMMIT на соединении сессии
createQuery(...).list()	построитель запросов session.select().from(...)
executeUpdate	session.insert(...), число изменённых строк – rowCount
System.out.println	console.log

3 Подключение к базе данных

Зависимости проекта – библиотека Drizzle ORM и драйвер pg – объявлены в файле package.json:

```
{
  "dependencies": {
    "drizzle-orm": "^0.45.4",
    "pg": "^8.23.1"
  }
}
```

Настройки подключения (адрес, порт, база данных, пользователь) заданы в файле config.ts; значения по умолчанию соответствуют отдельному кластеру PostgreSQL проекта и могут быть переопределены переменными окружения:

```
export const config = {
    host: process.env['PGHOST'] ?? '127.0.0.1',
    port: Number(process.env['PGPORT'] ?? 54329),
    database: process.env['PGDATABASE'] ?? 'velograd',
    user: process.env['PGUSER'] ?? 'postgres',
};
```

Отображение таблицы station на объект описано в файле schema.ts: столбцы, их типы и ограничения; тип NewStation выводится из описания и используется при вставке.

```
import { date, integer, numeric, pgTable, smallint, text } from 'drizzle-orm/pg-core';

export const station = pgTable('station', {
    id: integer().primaryKey().generatedByDefaultAsIdentity(),
    districtId: integer('district_id').notNull(),
    name: text().notNull(),
    address: text().notNull(),
    capacity: smallint().notNull(),
    latitude: numeric({ precision: 9, scale: 6 }).notNull(),
    longitude: numeric({ precision: 9, scale: 6 }).notNull(),
    openedOn: date('opened_on').notNull(),
});

export type NewStation = typeof station.$inferInsert;
```

4 Класс DAO

Класс DAO (файл DAO.ts) управляет подключением. Статическое поле sessionFactory – пул соединений, созданный из настроек config.ts; статическое поле session хранит текущую сессию. Метод getSession() проверяет, есть ли открытая сессия, и если её нет – берёт соединение из пула, создаёт на нём объект Drizzle, помещает его в поле session и возвращает. Метод begin() получает сессию и начинает транзакцию, commit() подтверждает её; метод close() освобождает соединение и закрывает пул.

```
import { drizzle } from 'drizzle-orm/node-postgres';
import { Pool, type PoolClient } from 'pg';
import { config } from './config.ts';

function open(client: PoolClient) {
    return drizzle(client);
}

export type Session = ReturnType<typeof open>;

export class DAO {
    static sessionFactory = new Pool(config);
    static session: Session | null = null;

    static async getSession(): Promise<Session> {
        if (DAO.session === null) DAO.session = open(await DAO.sessionFactory.connect());
        return DAO.session;
    }

    static async begin(): Promise<void> {
        const session = await DAO.getSession();
        await session.$client.query('BEGIN');
    }

    static async commit(): Promise<void> {
        const session = await DAO.getSession();
        await session.$client.query('COMMIT');
    }

    static async close(): Promise<void> {
        DAO.session?.$client.release();
        DAO.session = null;
        await DAO.sessionFactory.end();
    }
}
```

5 Класс TestSrvs

Класс TestSrvs – наследник DAO – содержит три метода (файл TestSrvs.ts):

– getName(id) – возвращает название станции с заданным идентификатором (один столбец, условие по первичному ключу);

– getNames(from, to) – возвращает список названий станций из промежутка идентификаторов, отсортированный по идентификатору;

– addStation(data) – создаёт запись; в отличие от чтения, метод обёрнут в пару begin() и commit(), чтобы результат сохранился, и использует вставку вместо выборки.

```
import { between, eq } from 'drizzle-orm';
import { DAO } from './DAO.ts';
import { type NewStation, station } from './schema.ts';

export class TestSrvs extends DAO {
    async getName(id: number): Promise<string | undefined> {
        const session = await DAO.getSession();
        const rows = await session.select({ name: station.name }).from(station).where(eq(station.id, id));
        return rows[0]?.name;
    }

    async getNames(from: number, to: number): Promise<string[]> {
        const session = await DAO.getSession();
        const rows = await session.select({ name: station.name }).from(station).where(between(station.id, from,
to)).orderBy(station.id);
        return rows.map((row) => row.name);
    }

    async addStation(data: NewStation): Promise<number> {
        await DAO.begin();
        const session = await DAO.getSession();
        const result = await session.insert(station).values(data);
        await DAO.commit();
        return result.rowCount ?? 0;
    }
}
```

6 Точка входа и результат

Класс Main (файл Main.ts) создаёт экземпляр TestSrvs, вызывает три метода, печатает результаты и закрывает пул соединений:

```
import { DAO } from './DAO.ts';
import { TestSrvs } from './TestSrvs.ts';

const srvs = new TestSrvs();

console.log(await srvs.getName(1));
console.log(await srvs.getNames(1, 5));
console.log(await srvs.addStation({
    districtId: 1,
    name: `Новая станция ${Date.now()}`,
    address: 'ул. Ленина, 1',
    capacity: 10,
    latitude: '55.750000',
    longitude: '37.600000',
    openedOn: '2026-11-05',
}));

await DAO.close();
```

Запуск – командой npm start (перед этим npm run pg:start). Результат показан на рисунке 1: название станции с идентификатором 1; список названий станций с идентификаторами от 1 до 5; число добавленных строк (1).

```
PS ...\pz4-orm-setup> npm start
Площадь Революции
[
  'Площадь Революции',
  'Городской сад',
  'Морской вокзал',
  'Приморский бульвар',
  'Заречье'
]
1
```

Рисунок 1 – Результат работы Main

Чтобы убедиться, что запись сохранена (выполнен COMMIT), она прочитана отдельным соединением psql (рисунок 2).

```
psql -d velograd -c "SELECT id, name, address, capacity FROM station WHERE id > 300 ORDER BY
id"
 id  |            name             |    address    | capacity
-----+-----------------------------+---------------+----------
 306 | Новая станция 1793831829215 | ул. Ленина, 1 |       10
(1 row)
```

Рисунок 2 – Добавленная станция в базе данных

7 Выводы

1 Связь приложения с базой данных настроена с помощью ORM Drizzle: параметры подключения вынесены в config.ts, отображение таблицы описано в schema.ts.

2 Класс DAO реализует управление сессиями: пул соединений как фабрика сессий, статическое поле текущей сессии, методы getSession(), begin() и commit().

3 Класс TestSrvs выполняет стандартные запросы: получение значения поля по идентификатору, получение списка значений из промежутка идентификаторов и вставку записи внутри транзакции; результат вставки подтверждён чтением из базы данных.